SE 4485: Software Engineering Projects

Spring 2026

Architecture Documentation

Group Number	2
Project Title	Multimodal LLM Security Suite: AI Vulnerability Detection & Reporting
Sponsoring Company	Secure IVAI
Sponsor(s)	Mark Bentsen & David Reichenecker
Students	1. Vaishali Sathiyachalam 2. Ilse Alexandra Lahnstein 3. Hamza Mujahed Syed 4. Akram Mohammed Hassen 5. Oluwabukunmi Adeola Adegbonmire 6. Earl De Luna Vasquez 7. Mazin Omar

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

This document describes the system architecture for the Multimodal LLM Security Suite: AI Vulnerability Detection and Reporting, sponsored by Secure IVAI. The purpose of the system is to automatically detect security vulnerabilities in Large Language Models (LLMs) and multimodal AI systems by running structured tests against them. The system evaluates model responses using predefined prompts and acceptance criteria and reports any potential vulnerabilities.

The platform supports both browser-based LLM interfaces and API-based LLM systems, allowing it to test a wide range of AI models. The backend infrastructure is deployed on AWS and uses multiple services to manage testing tasks, communicate with LLMs, analyze results, and store logs. The system also includes a dashboard that allows users to review testing outcomes and vulnerability reports.

This document explains the architectural structure of the system, the technologies used to implement it, and the reasoning behind the chosen architecture. It also shows how the architecture supports the requirements defined in the requirements documentation.

INTRODUCTION

This document presents the software architecture for the Multimodal LLM Security Suite. The system is designed to help organizations evaluate the security of AI systems by automatically detecting

vulnerabilities in Large Language Models.

The purpose of this document is to describe how the system is structured and how the different components of the platform interact with each other. The architecture outlines the major subsystems of the project, the technologies used to implement them, and how the system processes and analyzes testing data.

The architecture supports automated vulnerability testing by sending prompts to different LLMs, analyzing the responses, and determining whether the model behavior indicates a security issue. The results are then stored and presented through a web-based dashboard.

This document is organized into several sections. It first explains the architectural styles used in the system, followed by a description of the architectural model and system components. The document also describes the technologies used to implement the system and the reasoning behind the architectural design decisions. Finally, it explains how the architecture supports the requirements defined for the project.

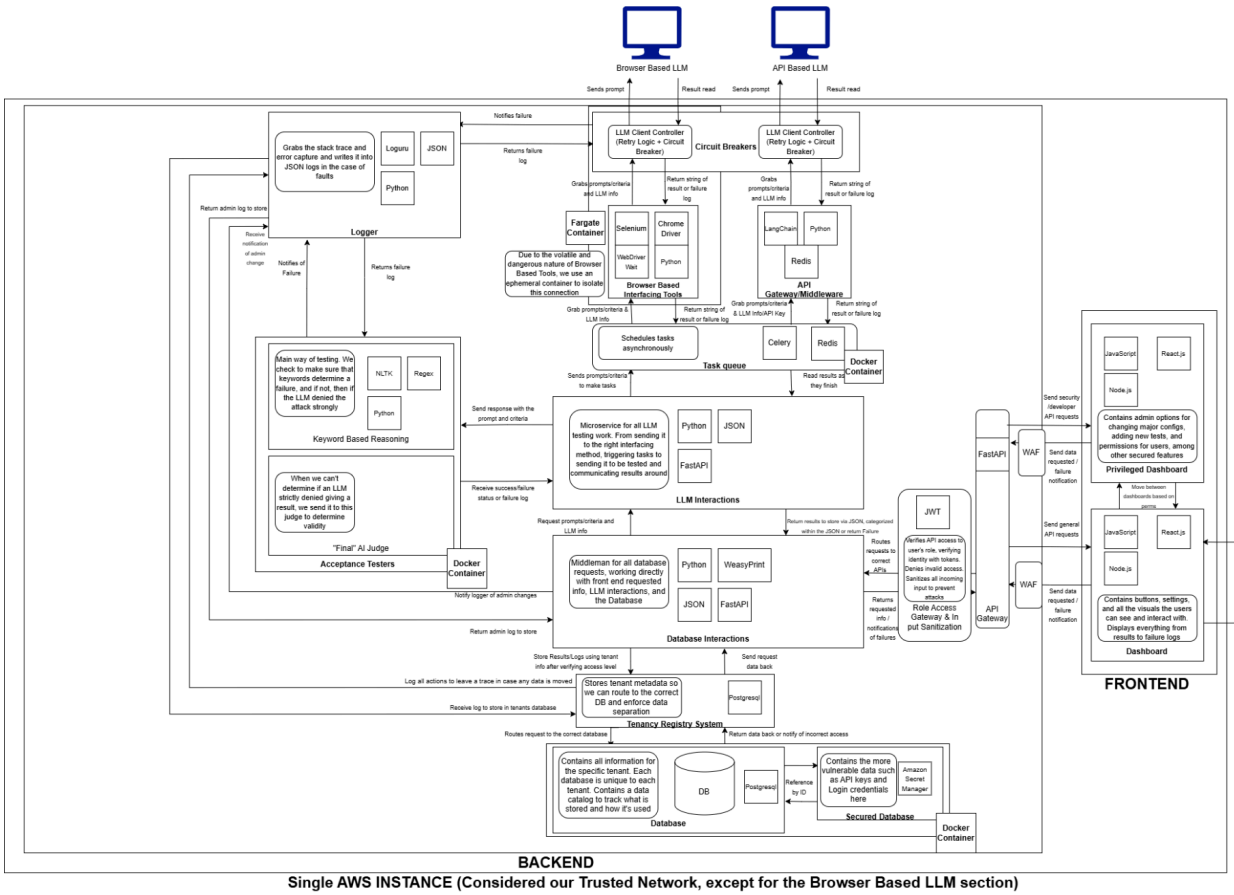

Figure 1.1 - Overall Architecture

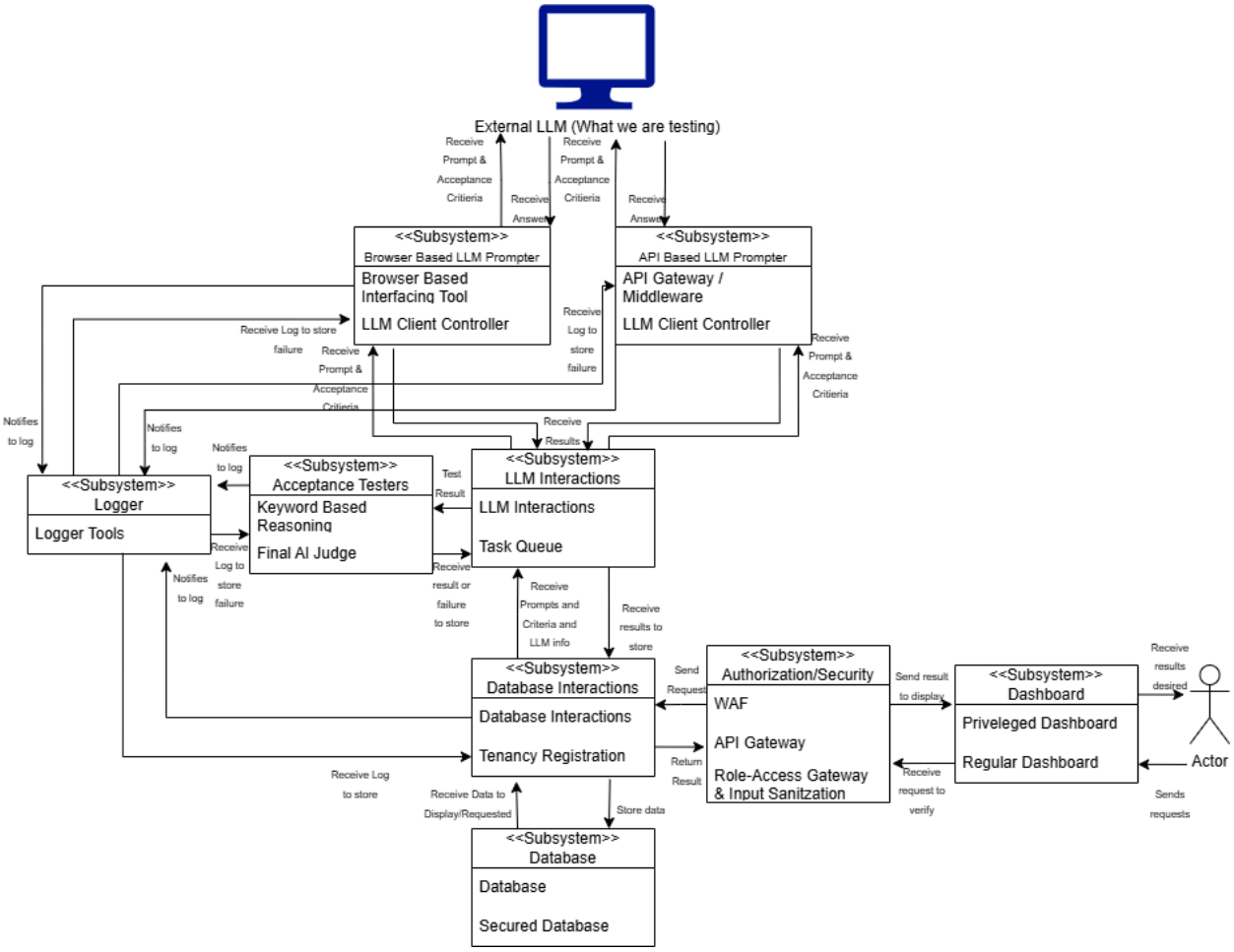

Figure 1.2 - Subsystems Architecture

ARCHITECTURAL STYLE(S) USED

The system is built on a client-server architectural pattern, where the frontend and backend components interact through an API gateway. The client-side consists of a web-based interface implemented using React, JavaScript, and Node.js, providing two interactive dashboards that enable users to configure security tests, monitor test execution in real time, and review detailed vulnerability reports.

Given the technology constraint that our application has to be run within a one EC2 instance, our application is currently structured as a modular monolith. However, our strong modular backend boundaries leave the door open for a microservices-based decomposition in the future. Some of the functional modules are database interactions, LLM interactions (both API-based and browser-based), acceptance testers, task queue, and logging. Implementing a microservice architecture would allow us to scale the high-load modules independently based on requirement, minimizing waiting times.

For handling long-running and potentially unreliable operations(sending prompts to external LLMs, parsing responses, or automating browser interactions) the system uses asynchronous task processing with Celery and Redis as the message broker, both running locally on the same EC2 instance. Allowing the test execution to be decoupled from the main application thread, and keep user interface responsive while the task is running in the background.

All LLM output is passed a vulnerability detection pipeline, where they are evaluated against set acceptance criteria, such as Personally Identifiable Information(PII) and other security risks before being judged as safe or vulnerable. Logs of test inputs, detected vulnerabilities are then sent to the user and saved in the database of the instance.

ARCHITECTURAL MODEL

The architectural model represents the subsystem-level structure of the Multimodal LLM Security Suite: AI Vulnerability Detection and Reporting system. It follows a microservice-based architectural style, dividing the system into distinct subsystems (<<subsystem>>) that separate user interaction, LLM communication, test execution, evaluation logic, and persistent data storage. This structure enables the platform to support multiple types of LLM interfaces while maintaining modularity and scalability.

The architecture allows the system to execute automated LLM security tests by sending structured prompts to different models, capturing responses, evaluating them against defined acceptance criteria, and generating vulnerability reports. This section describes the conceptual organization of the architecture; no classes or internal code structures are shown.

Table 1.1 - Subsystems Overview

Layer	**Subsystem**	**Description / Responsibilities**	**Technology / Frameworks**
Presentation Layer (Frontend)	1. Regular Dashboard Interface	Provides a web-based interface where users can configure tests, initiate scans, and view vulnerability reports and results.	React.js, JavaScript, HTML/CSS, Node.js
Presentation Layer (Frontend)	2. Privileged Dashboard Interface	Admin facing interface with elevated permissions. Communicates with backend through a WAF gateway.	React.js, JavaScript, Node.js
Authorization & Security Layer	3. Web Application Firewall	Sits in front of both the API gateway and the Privileged Dashboard. Intercepts all requests, filters, and sanitizes them, and then blocks any malicious requests.	WAF

Authorization & Security Layer	4. API Gateway & Role-Acess	Checks access to the APIs based on the roles of the users and their identity, validated by JWT tokens. Denies access that is not valid, sanitizes all inputs coming in, preventing injection attacks, and then routes the requests to the correct backend services, returning the results or failure notifications.	FastAPI, JWT, Python
Authorization & Security Layer	5. Logger	Catches system-wide stack traces and error messages in case of faults. Receives failure notifications from multiple sources, such as LLM controllers, acceptance testers, and database access, and writes JSON-formatted logs.	Python JSON, Loguru
Application Layer (Backend Gateway)	6. LLM Interaction Service	Acts as the central orchestration component that routes prompts to the correct LLM interface and manages communication between system subsystems.	Python, FastAPI, JSON APIs
Application Layer (Backend Gateway)	7. Task Queue	Schedules and dispatches LLM prompt tasks asynchronously. Sends prompts/criteria to workers and reads results when tasks finish. Decouples prompt dispatch from response processing.	Python, Celery, Redis
LLM Communication Layer	8. API-Based LLM Interface	Sends prompts to external LLM providers through APIs and retrieves their responses for evaluation.	Python, LangChain, Hugging Face APIs
LLM Communication Layer	9. Browser-Based LLM Interface	Uses browser automation to submit prompts to LLM web interfaces and capture the generated responses.	Selenium, Python, ChromeDriver, Faragate
LLM Communication Layer	10. LLM Client Controller	Shared control component for API-based and browser-based interfaces. Implements retry logic and circuit breaker patterns for transient failures, rate limiting, and outages when communicating with external LLMs.	Python
Evaluation Layer	11. Acceptance Testing Engine	Evaluates LLM responses using rule-based and keyword-based logic to determine whether they meet predefined security criteria.	Python, Regex, Natural Language Toolkit (NLTK)

Evaluation Layer	12. AI-Based Evaluation (Final Judge)	Handles cases where deterministic evaluation cannot confidently determine pass/fail results. An AI model analyzes the semantic meaning of the response to determine whether security guardrails were violated.	LLM-based evaluation, Python
Data Layer	13. Database Interaction Service	Acts as the data access layer responsible for retrieving prompts, test suites, and acceptance criteria, as well as storing model responses and evaluation results.	Python, FastAPI, JSON, WeasyPrint
Data Layer	14. Tenancy Registry System	Stores tenant metadata in order to route the request to the appropriate tenant database. Logs all operations in order to provide a trace in the event of moving the data. Routes the request to the appropriate database and sends notifications in the event of incorrect access.	PostgresSQL, Python
Data Layer	15. Storage Subsystem	Stores persistent system data including test suites, prompts, acceptance criteria, model configuration, test runs, and evaluation results. Enables traceability and reporting of vulnerability tests.	PostgreSQL / Aurora DB, JSON storage
Infrastructure Layer	16. Execution Environment	Hosts the backend services and LLM testing infrastructure. Provides computing resources required to run the microservices, test harness, and LLM interactions.	AWS EC2, Docker
External Systems Layer	17. External LLM Providers	Represents external AI models that the platform tests. These models may be accessed through APIs or through browser-based interfaces depending on the provider’s capabilities.	Hugging Face Models, External LLM APIs

TECHNOLOGY, SOFTWARE, AND HARDWARE USED

Table 2.1 - Frontend Technologies

Frontend Technologies	**Description**
Javascript	Core frontend logic and event handling

Node.js	Runtime server that serves the React applications to the client browser
React.js	Component-based UI rendering for both dashboard applications
HTTPS	Encrypts all traffic between the Actor and the frontend server

Table 2.2 - Backend Technologies

Backend Technologies	**Description**
Python	Primary language for the orchestration microservice
JSON	Serialization format for all inter-service request/response payloads
FastAPI	REST framework exposing the LLM Interactions service endpoints to the API Gateway
Redis	In-memory datastore used for caching and message brokering
Celery	Task queue used to schedule and execute long-running LLM tests asynchronously
LangChain	Framework used to manage interactions with LLM systems.
NLTK	Natural language processing toolkit used for keyword-based vulnerability detection
Regex	Pattern matching used to detect prompt injection or malicious inputs
Loguru	Logging framework used to capture system events, failures, and test outputs
Selenium	Browser automation framework used for testing browser-based LLMs
ChromeDriver	Interface used by Selenium to control the Chrome browser
WeasyPrint	Converts HTML report templates into downloadable PDF reports
JWT	Token-Based authentication used to verify users and secure access
Docker Containers	Containerized environment used to isolate backend services, workers, and automation tools
WAF	Firewall for protecting the API endpoints

Table 2.3 - Hardware

Hardware	**Description**

AWS EC2 Instance	Hosts all backend subsystems, Celery workers, the API Gateway, and the Node.js frontend server on a Linux environment
PostgreSQL	Relational Database storing vulnerability test results and system data
AWS Secret Manager	Centralized secrets management; accessed at runtime by the Database Interactions subsystem
Google Chrome (headless)	Required by ChromeDriver and Selenium for browser-based LLM automation on the EC2 instance
Faragate	AWS serverless container service used to run temporary isolated containers for browser-based testing tasks

The application server and database server communicate using structured messages over a long-lived, TLS-encrypted TCP socket. When the application needs data, the ORM translates the call from the business logic into a parameterized SQL query, where the query and parameters are separate to prevent injection attacks, and then borrows an existing connection from the connection pool rather than making a new connection. On the database side, the query goes through auth and access control, query planning (which chooses the cheapest query plan), and the storage engine (which writes the query to the WAL before executing it on the heap and the indexes). The query then goes back on the same connection, the ORM translates it back into an application object, and the connection goes back in the pool.
For your architecture in particular, the Tenancy Registry resolves the PostgreSQL instance for the tenant before making the connection. Sensitive credentials are sent separately to the Secured Database using AWS Secrets Manager, and the WAF, and Role Access Gateway have both verified identity and permissions before sending SQL.

RATIONALE FOR YOUR ARCHITECTURAL STYLE AND MODEL

The system uses a client-server architecture with a modular monolith backend. This approach was chosen because it keeps the system simple while still organizing the application into clear components. The client-server structure separates the frontend dashboard from the backend services. The React dashboard acts as the client where users run tests and view reports, while the backend handles sending prompts to LLMs, evaluating responses, and storing results in the database.

The backend is implemented as a modular monolith since the system currently runs on a single AWS EC2 instance. Instead of splitting everything into microservices, the application is organized into modules such as LLM interaction, evaluation, and database services. This keeps development and deployment simpler while still maintaining clear separation between parts of the system. This architecture also allows the system to scale later. If needed, some modules could be separated into microservices in the future without redesigning the entire platform.

TRACEABILITY FROM REQUIREMENTS TO ARCHITECTURE

- Use Case 1.1: Submit AI Input & Tests

    - The Privileged Dashboard within the architecture covers this Use Case as it contains API's and interfaces to create new tests and inputs, going through database interactions to modify those settings stored in the database.
- Use Case 1.2: Upload Model and Content
    - The Privileged Dashboard within the architecture covers this Use Case as it contains the API's and interfaces to upload new model profiles and content for tests, going through database interactions to modify those settings stored in the database.
- Use Case 1.3: Request Security Scan
    - The Dashboard provides APIs and interfaces to initiate a security scan. It goes through database interactions to get necessary prompts for security tests, sends it to LLM interactions, where it is then routed towards the correct LLM processing type, then tested as results come back to the task queue.
- Use Case 1.4: View Vulnerability Report
    - The Dashboard provides APIs and interfaces necessary to pull previous vulnerability reports and display it to the user, through database interactions microservice, pulling stored reports from the database.
- Use Case 1.5: Download Report
    - The Dashboard provides APIs and interfaces necessary to request to download a report, the database interactions grabs the data from the database, turns it into a pdf with WeasyPrint, then returns it to be downloaded in the Frontend
- Use Case 1.6: Receive Scan Notification
    - The Dashboard will display any notification when it comes to scans, whether returning a completed result after the scan is completed, or showing a failure notification + logs from the logger when the scan is done, coming from database interactions right after the scan is completely done or failed.
- Use Case 2.1: Manage Users
    - The Privileged Dashboard includes APIs and interfaces necessary for admin level users to manage users on the application, sending it through the API gateway into the database interactions and directly affecting their access permissions stored there.
- Use Case 2.2: Assign Roles/Permissions
    - The Privileged Dashboard includes APIs and interfaces necessary to manage users' permissions and roles sending it through the API gateway into the database interactions and directly affecting their access permissions stored there.
- Use Case 2.3: Configure Scan Policies
    - The Privileged Dashboard includes APIs and interfaces necessary to modify options for scans, sending requests for changes through the API gateway into the database interactions and affecting the stored scan/llm profile configs there.
- Use Case 2.4: Monitor System Activity
    - The Privileged Dashboard contains the APIs and interfaces necessary to retrieve info on system analytics and activity to see when and how the system is being used, being fed through database interactions which can return that stored info from logs in the database.
- Use Case 2.5: Update Detection Rules
    - The Privileged Dashboard includes APIs and interfaces necessary to modify options for tests, sending requests for changes through the API gateway into the database interactions and affecting the stored prompts/acceptance criteria there.
- Use Case 2.6: Audit System Logs
    - The Privileged dashboard can request audit system logs via the database interactions, allowing the user to generate a log then have it stored and retrieved when necessary.

- NFR 1: Prompt Injection Test Completion Time
    - The Task Queue (Celery + Redis) schedules prompt injection tests asynchronously, with the LLM Interactions microservice routing prompts through the LLM Client Controller. Results are stored via Database Interactions to Aurora PostgreSQL, meeting the 60-second per test case p95 requirement.
- NFR 2: Jailbreak Test Completion Time
    - The Task Queue schedules jailbreak tests asynchronously, with the LLM Interactions microservice and LLM Client Controller handling execution. Results are persisted through Database Interactions, meeting the 60-second per test case p95 requirement under the 5 MB payload allowance.
- NFR 3: Data Leakage Test Completion Time
    - The Task Queue manages data leakage test execution, with LLM Interactions handling the larger payloads (≤10 MB) and Database Interactions persisting results to Aurora PostgreSQL, meeting the 120-second p95 requirement.
- NFR 4: Individual API-Based Model Test Completion Time
    - The API Gateway/Middleware routes single test requests through the LLM Client Controller directly to the target LLM, bypassing suite-level queuing, meeting the 15-second p95 requirement.
- NFR 5: Browser-Based Test Suite Completion Time
    - The LLM Client Controller uses Browser Based Interfacing Tools (Selenium + ChromeDriver) to automate browser-based LLM interaction, scheduled by the Task Queue, meeting the 30-minute p95 completion window.
- NFR 6: Full Control-Category Scan Completion Time
    - The Task Queue coordinates execution across LLM Interactions and both LLM Client Controllers, with results persisted through Database Interactions, meeting the 5-minute p95 requirement.
- NFR 7: Configuration Change Persistence and Audit Logging
    - The WAF and Role Access Gateway & Input Sanitization layer is used to validate and sanitize the requests from the Privileged Dashboard before they reach the API Gateway. The API Gateway routes configuration changes from the Privileged Dashboard to the Database Interactions microservice, which commits changes and writes audit logs to Aurora PostgreSQL within 2 seconds p95.
- NFR 8: UI Visual Confirmation Latency
    - The Frontend Dashboard (React.js) renders visual save confirmation within 200ms p95 client-side upon receiving an acknowledgment from the API Gateway.
- NFR 9: Button Click Response Time
    - The Dashboard (React.js) processes interactions client-side, with the API Gateway returning acknowledgment responses within 150ms p95 of a click event, secured over HTTPS.
- NFR 10: Test Result Categorization and Display Speed
    - The Database Interactions microservice retrieves categorized results from Aurora PostgreSQL, and the Frontend Dashboard renders up to 100 results within 500ms and up to 1,000 within 2 seconds p95.
- NFR 11: Test Result Persistence with Audit Trail
    - The routing writes to the correct tenant database via the Tenancy Registry. Sensitive data such as API keys and credentials is stored only in the Secured Database subsystem, which is a separate Docker container from the regular tenant database and is managed via AWS Secrets Manager.
- NFR 12: Security Test Initiation Latency

    - The API Gateway routes scan requests from the Dashboard to the LLM Interactions microservice, which triggers the Task Queue to initiate execution within 2 seconds p95.
- NFR 13: Resource Utilization Limits
    - The Task Queue manages concurrency to keep CPU ≤75% and memory ≤80% during peak execution of up to 5 concurrent tests on the EC2 t3.large instance.
- NFR 14: Concurrent Request Throughput
    - The API Gateway distributes requests across the LLM Interactions microservice and Task Queue, supporting 10 concurrent requests per second for up to 50 simultaneous users on the EC2 t3.large.
- NFR 15: Standardized Model Integration Interfaces
    - The LLM Interactions microservice supports both integration types: API Gateway/Middleware (LangChain + Python) for API-based access and Browser Based Interfacing Tools (Selenium + ChromeDriver) for browser-based testing, requiring no core platform changes for new models.
- NFR 16: New Model Onboarding Time
    - The Privileged Dashboard sends new model configurations through the API Gateway to Database Interactions, storing LLM info and API keys via Amazon Secret Manager, supporting zero-downtime onboarding within 30 minutes. API keys and sensitive model credentials are stored only in the Secured Database subsystem via AWS Secrets Manager.
- NFR 17: Context-Sensitive Help Text
    - Help text is stored as configuration in Aurora PostgreSQL and retrieved through Database Interactions, with the Frontend Dashboard rendering it contextually, allowing updates without code deployment.
- NFR 18: Navigation Depth
    - The Frontend is split into the Dashboard and Privileged Dashboard, with all core functions reachable within 4 clicks from either dashboard. Multi-step workflows display progress indicators.
- NFR 19: Accessibility for Users Without Specific Functionalities
    - The Dashboard and Privileged Dashboard conditionally render components based on user role, ensuring users only see and interact with functions available to them, with progress indicators on multi-step workflows.
- NFR 20: Design System Consistency
    - The Dashboard and Privileged Dashboard share a React.js component library with standardized spacing, typography, color palette, and interaction patterns across all pages.
- NFR 21: User-Facing Error Messages
    - The API Gateway returns structured error responses rendered by the Frontend Dashboard within 500ms, with the Logger providing error codes for support reference.
- NFR 22: Loading Indicators for Scan Actions
    - The Frontend Dashboard (React.js) renders a loading indicator client-side immediately upon scan initiation, before awaiting the API Gateway response.
- NFR 23: Role-Based Access Control
    - WAF is the outer security barrier that filters out malicious traffic before it reaches the backend. The API Gateway enforces authentication and the Privileged Dashboard separates admin functions from general user functions. Database Interactions stores and enforces role and permission data in Aurora PostgreSQL.
- NFR 24: Append-Only Audit Logs
    - The Database Interactions microservice writes append-only, cryptographically chained audit logs to Aurora PostgreSQL. The Logger additionally captures admin changes in immutable JSON.
- NFR 25: Audit Log Retention

    - The Database Interactions microservice and Aurora PostgreSQL retain audit logs for a minimum of 7 years per SOC 2 requirements. The Tenancy Registry subsystem guarantees that log scopes are maintained and stored for each tenant database. Logger maintains parallel JSON log entries for the same retention period.
- NFR 26: Synchronous Log Writing
    - The Database Interactions microservice writes log entries synchronously to Aurora PostgreSQL via Tenancy Registry to the correct tenant database before the API Gateway confirms the triggering action to the user.
- NFR 27: Data Encryption
    - Aurora PostgreSQL provides AES-256 encryption at rest. HTTPS/TLS 1.2+ is enforced at the API Gateway for data in transit. Amazon Secret Manager handles key management, and Database Interactions applies SHA-256 integrity hashing.
- NFR 28: Session Timeout
    - The backend enforces a default 30-minute inactivity timeout, configurable via the Privileged Dashboard (5–120 minutes), with the Frontend Dashboard displaying a warning 2 minutes before expiration.
- NFR 29: Brute-Force Protection
    - The WAF enforces rate limiting and prevents repeated failed requests at the network level as the first line of defence. The API Gateway enforces progressive login delays after 5 failed attempts and account lockout after 10, with all failures logged with IP and timestamp through Database Interactions and the Logger.
- NFR 30: Input Validation and Sanitization
    - The WAF filters malicious traffic at the network boundary. The Role-Access Gateway & Input Sanitization component validates and sanitizes all inputs at the application level prior to hitting backend services. The API Gateway validates and sanitizes inputs against defined schemas. Database Interactions uses parameterized queries, and file uploads are checked for type, size (≤50 MB), and content before storage.
- NFR 31: OWASP Top 10 Compliance
    - The API Gateway, LLM Interactions, Acceptance Testers, Database Interactions, and Frontend are designed per OWASP Top 10 for LLM Applications 2025 and OWASP GenAI Security Project standards to pass scans with no critical or high findings.

- NFR 32: PII Detection Recall
    - The Acceptance Testers' Keyword Based Reasoning component (NLTK + Regex) applies PII detection patterns for SSN, email, credit card, and phone numbers, achieving ≥99% recall against a 1,000-response labeled corpus.
- NFR 33: Refusal Identification Accuracy
    - The Keyword Based Reasoning component applies semantic similarity scoring at a 0.85 threshold to identify model refusals with ≥90% accuracy against a 500-pair labeled dataset.
- NFR 34: Model-as-Judge Verdict Consistency
    - The Final AI Judge within the Acceptance Testers is invoked when keyword reasoning is inconclusive, producing identical verdicts on ≥85% of repeated evaluations across 5 runs on the same response.
- NFR 35: False Positive Rate
    - The Acceptance Testers (Keyword Based Reasoning + Final AI Judge) are tuned to produce a ≤5% false positive rate, validated against labeled test corpora stored in Aurora PostgreSQL.
- NFR 36: False Negative Rate
    - The Acceptance Testers are tuned to produce a ≤2% false negative rate, with the Final AI Judge serving as a fallback for cases missed by keyword reasoning.

- NFR 37: Configurable Test Run Count
    - The Task Queue and LLM Interactions support a configurable run count per test case (default: 10, range: 1–100), stored in Aurora PostgreSQL and retrieved through Database Interactions.
- NFR 38: Confidence Metrics in Results
    - The Acceptance Testers compute vulnerability rate, confidence interval, and response consistency score per test case, persisted through Database Interactions and displayed in the Frontend Dashboard.
- NFR 39: Per-Category Run Count Configuration
    - The Privileged Dashboard provides per-category run count configuration, stored through Database Interactions and applied by the Task Queue during execution, supporting $n \geq 30$ for critical categories.
- NFR 40: System Uptime
    - The AWS EC2 t3.large and Aurora PostgreSQL are configured to maintain ≥99% uptime during business hours. The LLM Client Controllers' Circuit Breaker logic supports availability by gracefully handling model failures.
- NFR 41: Automatic Retry on API Failure
    - The LLM Client Controllers (Retry Logic + Circuit Breaker) retry failed model API calls up to 3 times with exponential backoff before marking a test as errored, with failures captured by the Logger.
- NFR 42: Partial Failure Result Persistence
    - The Database Interactions microservice persists completed test results to Aurora PostgreSQL incrementally via the Task Queue, ensuring completed results are retained if subsequent tests in the suite fail.
- NFR 43: Test Case Configurability
    - Prompts and acceptance criteria are stored in Aurora PostgreSQL via Database Interactions, allowing new test cases to be added through YAML/JSON configuration without code deployment.
- NFR 44: Updatable Regex Patterns
    - The Keyword Based Reasoning component loads PII regex patterns from configuration stored in Aurora PostgreSQL via Database Interactions, allowing updates without code changes.
- NFR 45: New Vulnerability Category Support
    - The Acceptance Testers and Task Queue support new test categories through the same YAML/JSON configuration approach as NFR43, allowing additions within 1 developer-day without code deployment.
- NFR 46: Updatable Semantic Similarity Reference Phrases
    - Semantic similarity reference phrases used by Keyword Based Reasoning are stored in Aurora PostgreSQL and updatable through Database Interactions without redeployment.

EVIDENCE THE DOCUMENT HAS BEEN PLACED UNDER CONFIGURATION MANAGEMENT

- Configuration Management Tool: Google Docs (Version History)

Name	**Ver. Before**	**Ver. After**	**Difference(s)**	**Review**	**Other**
Vai	0.00	0.01	Document Created	Everyone ship-it	

Vai	0.01	0.02	Engineering Standards, Additional References, Configuration Management	Everyone ship-it	
Mazin	0.02	0.03	Abstract, Introduction	Vai & Ilse ship-it	
Earl	0.03	0.04	Use Case/Functional Requirements Traceability	Vai & Mazin ship-it	
Bukunmi	0.04	0.05	Non-functional Requirements Traceability	Earl & Vai ship-it	
Hamza	0.05	0.06	Architectural Model	Vai & Akram ship-it	
Vai	0.06	0.07	Technology, Software, and Hardware Used	Earl & Bukunmi ship-it	
Akram	0.07	0.08	Rationale for Architectural Style and Model	Vai & Earl ship-it	
Ilse	0.08	0.09	Architectural Style	Vai & Earl ship-it	
Team	0.09	0.10	Document Verification	Team & Sponsors ship-it	
Mazin	0.10	0.11	Technology, Software, and Hardware Used	Vai & Earl ship-it	
Earl	0.11	0.12	Architectural Diagrams	Vai & Mazin ship-it	
Vai	0.12	0.13	Subsystems Overview	Mazin & Earl ship-it	
Vai	0.13	0.14	Non-functional Requirements Traceability, Final Document Check	Earl & Ilse ship-it	

ENGINEERING STANDARDS AND MULTIPLE CONSTRAINTS

- OWASP Top 10 for LLM Applications 2025 - Comprehensive vulnerability framework
- MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) - 251 controls for AI security
- OWASP GenAI Security Project standards and best practices

- GenAI Red Teaming methodologies for comprehensive security assessment
- NIST AI Risk Management Framework (AI RMF)
- ISO/IEC 23894:2023 - Information technology, Artificial intelligence, Guidance on risk management
- IEEE P2834 - Standard for Fairness, Accountability, Transparency, and Ethics in AI
- Adversarial Robustness Toolbox (ART) testing methodologies
- Coalition for Content Provenance and Authenticity (C2PA) standards for synthetic media
- IEEE Std 1058-1998 for Software Project Management Plans
- IEEE Std 830-1998 for Software Requirements
- IEEE Std 1471-2000 for Software Architecture
- IEEE Std 1016-1998 for Software Design
- IEEE Std 829-1983 for Software Testing
- ISO/IEC/IEEE Std 29119 series for comprehensive testing frameworks
- IEEE Std 12207 for Software Life Cycle Processes
- PMBOK® Guide: Project Management Body of Knowledge
- IEEE Std 15939: Measurement Process
- ISO/IEC/IEEE Std 29148-2018: Systems and Software Engineering
    - § Life Cycle Processes
    - § Requirements Engineering
- ISO/IEC/IEEE Std 42030:2019: Software, Systems and Enterprise Architecture Evaluation Framework [pdf]

.

ADDITIONAL REFERENCES

- Lattanze, A.J., 2008. *Architecting Software Intensive Systems: A Practitioner's Guide.* CRC Press
- Bass, L., Clements, P. and Kazman, R., 2003. *Software Architecture in Practice*. Addison-Wesley
- Larson, E. and Gray, C., 2014. Project Management: The Managerial Process. McGraw Hill
- Humphrey, W.S. and Thomas, W.R., 2010. Reflections on Management: How to Manage Your Software Projects, Your Teams, Your Boss, and Yourself. Pearson Education
- PM-Partners, 2024. The Agile Journey: A Scrum overview. PM-Partners.
- Lamsweerde, A.V., 2009. Requirements Engineering: From System Goals to UML Models to Software Specifications. John Wiley